

Fixed Bases to RBF Networks and Neural Networks, and a bit on RKHS

1 From Basis Expansions to Neural Networks

Before introducing RKHS theory, it is helpful to understand how modern machine learning methods arise from classical basis expansions.

1.1 Univariate Basis Expansions

Suppose $x \in \mathbb{R}$ is a scalar predictor and the regression relationship is

$$y_i = f(x_i) + \varepsilon_i, \quad i = 1, \dots, n,$$

where f is an unknown smooth function. A standard nonparametric strategy is to approximate f by a finite basis expansion:

$$f(x) = \sum_{j=1}^N \beta_j B_j(x),$$

where $\{B_j(x)\}_{j=1}^N$ is a chosen collection of basis functions and $\beta_1, \dots, \beta_N$ are unknown coefficients.

The basis functions determine the function class used for approximation. Common examples include:

- **Polynomial bases:**

$$B_j(x) = x^j, \quad j = 0, 1, \dots, N.$$

These produce global polynomial approximations. They are simple and easy to interpret, but high-degree polynomials can be numerically unstable and may behave poorly near the boundaries.

- **Fourier bases:**

$$B_{2j-1}(x) = \sin(2\pi jx), \quad B_{2j}(x) = \cos(2\pi jx), \quad j = 1, 2, \dots$$

These are well suited for periodic or approximately periodic signals. Since each basis function is global, Fourier expansions are effective for smooth oscillatory structure but less effective for highly localized features.

- **Spline bases:** Here $B_j(x)$ are spline basis functions, typically B-splines or truncated power bases, defined relative to a knot sequence. Splines are piecewise polynomials joined smoothly at knots, and hence provide flexible local approximation while maintaining smoothness.

- **Wavelet bases:** Here $B_j(x)$ are wavelet basis functions, such as Haar or Daubechies wavelets. Wavelets are localized in both position and scale, so they are especially useful for functions having local irregularities, jumps, or spatially inhomogeneous smoothness.

After fixing the basis, estimation reduces to finding the coefficients $\boldsymbol{\beta} = (\beta_1, \dots, \beta_N)^\top$. A common approach is penalized least squares:

$$\min_{\boldsymbol{\beta}} \sum_{i=1}^n \left(y_i - \sum_{j=1}^N \beta_j B_j(x_i) \right)^2 + \lambda \boldsymbol{\beta}^\top \boldsymbol{\Omega} \boldsymbol{\beta}.$$

The first term measures goodness of fit, while the penalty term controls roughness or complexity. The matrix $\boldsymbol{\Omega}$ depends on the basis and the desired notion of smoothness. For example, in spline smoothing, $\boldsymbol{\Omega}$ is often derived from an integrated squared derivative penalty such as

$$\int \{f''(x)\}^2 dx.$$

The tuning parameter $\lambda \geq 0$ balances fidelity to the data against smoothness. When $\lambda = 0$, the model interpolates more closely to the data; when λ is large, the fitted function becomes smoother and less variable.

An important point is that the model is *linear in the coefficients* $\boldsymbol{\beta}$, since

$$f(x) = \boldsymbol{\beta}^\top \mathbf{B}(x), \quad \mathbf{B}(x) = (B_1(x), \dots, B_N(x))^\top,$$

even though $f(x)$ can be highly nonlinear as a function of x . This linear-in-parameters representation is what makes basis expansions computationally convenient.

Thus, univariate basis expansions provide a flexible but structured framework for nonparametric regression. They form a classical foundation for smoothing, function estimation, and regularization.

1.2 Multivariate Basis Expansions

Now let $\mathbf{x} = (x_1, \dots, x_d)^\top \in \mathbb{R}^d$ be a multivariate predictor. The same general idea extends to

$$f(\mathbf{x}) = \sum_{j=1}^N \beta_j \phi_j(\mathbf{x}),$$

where $\phi_1, \dots, \phi_N$ are basis functions defined on $\mathbb{R}^d$.

A particularly common construction is to build multivariate basis functions from lower-dimensional ones. Examples include:

- **Tensor-product splines:** For $d = 2$,

$$\phi_{jk}(x_1, x_2) = B_j(x_1)B_k(x_2),$$

where B_j and B_k are univariate spline basis functions. More generally, for d predictors,

$$\phi_{j_1, \dots, j_d}(\mathbf{x}) = \prod_{\ell=1}^d B_{j_\ell}(x_\ell).$$

Tensor-product constructions are highly flexible and preserve the local structure of spline bases, but the number of basis functions can grow rapidly with dimension.

- **Multivariate polynomial bases:** These include monomials such as

$$x_1^{a_1} x_2^{a_2} \cdots x_d^{a_d},$$

with nonnegative integers $a_1, \dots, a_d$. Such bases are conceptually simple but can become large and ill-conditioned for moderate degree and dimension.

- **Multivariate Fourier bases:** These are built from products of univariate sine and cosine terms, for example

$$\sin(2\pi j x_1) \cos(2\pi k x_2),$$

or more general trigonometric combinations. They are natural for periodic multivariate functions.

- **Multivariate wavelets:** These are typically formed by tensor products of univariate wavelets and scaling functions. They provide multiscale, localized representations for multivariate signals and surfaces.

As in the univariate case, the model remains linear in the coefficients:

$$f(\mathbf{x}) = \boldsymbol{\beta}^\top \boldsymbol{\phi}(\mathbf{x}),$$

where

$$\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_N(\mathbf{x}))^\top.$$

This is often called a **feature map representation**, because the basis functions map the original input $\mathbf{x}$ into a new feature vector $\boldsymbol{\phi}(\mathbf{x})$, after which regression is linear in that transformed feature space.

The main challenge is that N , the number of basis functions needed for accurate approximation, often grows very quickly with the dimension d . For instance, if each coordinate uses K basis functions, then a full tensor-product construction may require K^d basis terms. This exponential growth is one form of the *curse of dimensionality*, and it is a central reason why multivariate nonparametric regression is substantially harder than the univariate case.

In practice, this difficulty motivates structured alternatives, such as additive models, low-rank tensor decompositions, sparse basis expansions, or learned feature representations.

Fixed versus adaptive bases. The spline, Fourier, and classical wavelet bases described above are *fixed* once the knots, frequencies, or wavelet family are chosen. That is, their functional forms are specified in advance and only the coefficients are estimated from the data.

In contrast to splines or wavelets, the basis functions considered from here onward will include unknown, learnable parameters that are themselves estimated from the data. Thus, the basis functions are *data-adaptive* rather than fixed. This allows the representation to adjust its shape, location, and other features to better match the observed regression structure.

1.3 Radial Basis Function Models

Radial basis function models use localized basis functions:

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma^2}\right).$$

These models avoid tensor-product explosion because the basis functions depend on distance rather than coordinate-wise interactions.

RBF models are flexible approximators and naturally extend to high-dimensional inputs.

They also provide the first connection to RKHS methods, because the Gaussian kernel is exactly the covariance function of Gaussian RKHS regression.

1.4 RBF Networks

RBF models become neural networks when the centers $\mathbf{c}_j$ are treated as parameters:

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \phi(\|\mathbf{x} - \mathbf{c}_j\|).$$

Training now involves estimating:

$$\{w_j, \mathbf{c}_j\}_{j=1}^M.$$

This makes the model nonlinear in the parameters, unlike classical basis expansions.

RBF networks therefore represent a transition between

$$\text{fixed bases} \quad \longrightarrow \quad \text{learned bases}.$$

1.5 RBF Networks and Gaussian Mixture Models

RBF models become neural networks when the centers $\mathbf{c}_j$ are treated as parameters:

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \phi(\|\mathbf{x} - \mathbf{c}_j\|).$$

Training now involves estimating:

$$\{w_j, \mathbf{c}_j\}_{j=1}^M.$$

This makes the model nonlinear in the parameters, unlike classical basis expansions.

RBF networks therefore represent a transition between

$$\text{fixed bases} \quad \longrightarrow \quad \text{learned bases}.$$

Connection with Gaussian Mixture Models

When the radial basis function is Gaussian,

$$\phi(\|\mathbf{x} - \mathbf{c}_j\|) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma^2}\right),$$

the RBF network takes the form

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma^2}\right).$$

This structure closely resembles a Gaussian mixture model:

$$p(\mathbf{x}) = \sum_{j=1}^M \pi_j \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j),$$

where

- $\boldsymbol{\mu}_j$ are component means,
- $\boldsymbol{\Sigma}_j$ are covariance matrices,
- π_j are mixture weights.

Each Gaussian mixture component

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\top \boldsymbol{\Sigma}_j^{-1}(\mathbf{x} - \boldsymbol{\mu}_j)\right)$$

has exactly the same functional form as a Gaussian radial basis function.

Thus an RBF network can be viewed as a **weighted sum of Gaussian components**:

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \phi_j(\mathbf{x}).$$

Regression vs Density Estimation

The key difference is the objective:

	Gaussian Mixture Model	RBF Network
Goal	Density estimation	Regression
Output	$p(\mathbf{x})$	$f(\mathbf{x})$
Weights	$\pi_j \geq 0$, sum to 1	arbitrary w_j
Training	EM algorithm	least squares / SGD

From this viewpoint:

Gaussian mixtures $\longrightarrow$ RBF networks

represent the transition from

unsupervised density modeling $\longrightarrow$ supervised function approximation.

The Connection

Gaussian mixture models and RBF networks use the same Gaussian building blocks. Gaussian mixtures model densities, while RBF networks use the same components to approximate regression functions.

1.6 Feedforward Neural Networks

General neural networks replace radial functions with arbitrary nonlinear activations:

$$f(\mathbf{x}) = \sum_{j=1}^M a_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j).$$

Here

- $\mathbf{w}_j^\top \mathbf{x} + b_j$ defines a learned feature,
- $\sigma(\cdot)$ is a nonlinear activation function, (for some choices of $\sigma(\cdot)$, $\mathbf{w}_j$ and b , one can go back to radial bases).
- a_j are output weights.

Thus neural networks can be interpreted as **adaptive basis expansions**.

The basis functions

$$\phi_j(\mathbf{x}) = \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

are learned from the data rather than fixed in advance.

1.7 Deep Neural Networks

Deep neural networks extend this idea by composing multiple layers of learned features:

$$f(\mathbf{x}) = \mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\cdots \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots) + \mathbf{b}_{L-1}) + \mathbf{b}_L).$$

Each layer produces a new feature representation, leading to hierarchical function approximation.

Thus we obtain the progression

fixed basis expansion $\rightarrow$ RBF models $\rightarrow$ shallow neural networks $\rightarrow$ deep neural networks.

Key Insight

Neural networks are best understood as adaptive multivariate basis expansions. RKHS theory provides the mathematical framework that connects these basis expansions to kernel methods.

2 RKHS Motivation

Modern machine learning methods — kernel machines, radial basis function (RBF) networks, and neural networks — are often presented as fundamentally different approaches. In reality, they share a common mathematical foundation based on *function approximation*.

In many learning problems we observe data

$$(x_1, y_1), \dots, (x_n, y_n),$$

and the goal is to estimate an unknown function

$$f : \mathcal{X} \rightarrow \mathbb{R}$$

that predicts y from x .

Unlike classical parametric regression, where we estimate a finite vector of parameters $\beta \in \mathbb{R}^p$, modern learning methods estimate an entire function f . Thus the parameter space is typically *infinite dimensional*.

Many learning algorithms can be written in the general form

$$\min_{f \in \mathcal{F}} \left\{ \sum_{i=1}^n L(y_i, f(x_i)) + \lambda J(f) \right\},$$

where

- $L(y, f(x))$ measures goodness of fit,
- $J(f)$ measures complexity or roughness,
- λ controls regularization,
- $\mathcal{F}$ is a space of functions.

Many learning algorithms are optimization problems over function spaces. Reproducing Kernel Hilbert Space (RKHS) theory provides a mathematical framework that unifies kernel methods, RBF networks, and many neural network constructions.

Understanding RKHS theory helps explain:

- why kernels work,
- why regularization is necessary,
- why solutions often have simple finite representations,
- how classical smoothing methods relate to modern machine learning.

2.1 Infinite-Width Neural Networks and Gaussian Process Limits

Consider a single-hidden-layer fully connected neural network of width m

$$f_m(x) = \frac{1}{\sqrt{m}} \sum_{j=1}^m a_j \sigma(w_j^\top x + b_j),$$

where the parameters satisfy

$$w_j \sim N(0, \Sigma_w), \quad b_j \sim N(0, \sigma_b^2), \quad a_j \sim N(0, \sigma_a^2),$$

independently for $j = 1, \dots, m$. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a measurable activation function satisfying

$$\mathbb{E}[\sigma(Z)^2] < \infty, \quad Z \sim N(0, 1).$$

Theorem 2.1 (Infinite-width neural network limit). *Let $x_1, \dots, x_n \in \mathbb{R}^d$ be fixed inputs. As the width $m \rightarrow \infty$,*

$$(f_m(x_1), \dots, f_m(x_n)) \xrightarrow{d} \mathcal{N}(0, K),$$

where the covariance matrix K has entries

$$K(x, x') = \sigma_a^2 \mathbb{E}_{w,b} [\sigma(w^\top x + b) \sigma(w^\top x' + b)].$$

Consequently, the random function $f_m(\cdot)$ converges in finite-dimensional distributions to a Gaussian process

$$f(\cdot) \sim \mathcal{GP}(0, K).$$

Interpretation. The infinite-width neural network induces a Gaussian process prior on functions with covariance kernel

$$K(x, x') = \mathbb{E}[\phi(x)\phi(x')], \quad \phi(x) = \sigma(w^\top x + b).$$

The corresponding function space is the reproducing kernel Hilbert space (RKHS) generated by K .

Example 1: Linear activation. If $\sigma(z) = z$, then

$$K(x, x') = x^\top x'.$$

Thus the infinite-width network reduces to a linear kernel model.

Example 2: ReLU activation. For $\sigma(z) = \max(0, z)$ and $w \sim N(0, I)$,

$$K(x, x') = \frac{\|x\| \|x'\|}{2\pi} \left(\sin \theta + (\pi - \theta) \cos \theta \right),$$

where

$$\theta = \cos^{-1} \left(\frac{x^\top x'}{\|x\| \|x'\|} \right).$$

This kernel is known as the *arc-cosine kernel*.

Example 3: Tanh activation. If $\sigma(z) = \tanh(z)$, then

$$K(x, x') = \mathbb{E}[\tanh(u) \tanh(v)],$$

where

$$(u, v) \sim N \left(0, \begin{pmatrix} x^\top x & x^\top x' \\ x^\top x' & x'^\top x' \end{pmatrix} \right).$$

Deep networks. For a depth L fully connected network, the covariance kernel satisfies the recursion

$$K^{(\ell+1)}(x, x') = \sigma_a^2 \mathbb{E}_{(u,v)}[\sigma(u)\sigma(v)] + \sigma_b^2,$$

where

$$(u, v) \sim N\left(0, \begin{pmatrix} K^{(\ell)}(x, x) & K^{(\ell)}(x, x') \\ K^{(\ell)}(x, x') & K^{(\ell)}(x', x') \end{pmatrix}\right).$$

References. Early results connecting infinitely wide neural networks to Gaussian processes appear in [Neal, 1996, Williams, 1997]. Explicit neural network kernels were studied in [Cho and Saul, 2009], and the deep network Gaussian process limit was established in [Lee et al., 2018].

3 Gaussian Processes and Their Connection to RKHS

Gaussian processes provide a flexible nonparametric framework for modeling unknown functions. They are closely connected to kernel methods and reproducing kernel Hilbert spaces (RKHS), which play a central role in modern statistical learning theory.

3.1 Gaussian Processes

Definition 3.1. A stochastic process $\{f(x) : x \in \mathcal{X}\}$ is called a *Gaussian process* if for every finite collection $x_1, \dots, x_n \in \mathcal{X}$ the vector

$$(f(x_1), \dots, f(x_n))$$

has a multivariate normal distribution.

A Gaussian process is completely determined by its mean function

$$m(x) = \mathbb{E}[f(x)]$$

and covariance function

$$K(x, x') = \text{Cov}(f(x), f(x')).$$

We write

$$f(x) \sim GP(m(x), K(x, x')).$$

Throughout most applications the mean is taken to be zero, so the GP prior becomes

$$f(x) \sim GP(0, K(x, x')).$$

3.2 Positive Definite Kernels and RKHS

The covariance function K must be a *positive definite kernel*. That is, for any $x_1, \dots, x_n$ and any vector $c_1, \dots, c_n$,

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(x_i, x_j) \geq 0.$$

Every positive definite kernel defines a reproducing kernel Hilbert space (RKHS).

Theorem 3.1 (Moore-Aronszajn). *For every positive definite kernel K on $\mathcal{X}$ there exists a unique Hilbert space $\mathcal{H}_K$ of functions on $\mathcal{X}$ such that*

$$f(x) = \langle f, K(\cdot, x) \rangle_{\mathcal{H}_K}$$

for all $f \in \mathcal{H}_K$.

The space $\mathcal{H}_K$ is called the RKHS associated with kernel K .

Thus the covariance function of a Gaussian process simultaneously defines

- a stochastic process prior on functions
- a deterministic function space $\mathcal{H}_K$.

3.3 Sampling from a Gaussian Process

Let $x_1, \dots, x_n$ be inputs. Define the kernel matrix

$$K_{ij} = K(x_i, x_j).$$

Then

$$(f(x_1), \dots, f(x_n))^{\top} \sim N(0, K).$$

Hence sampling a Gaussian process reduces to sampling from a multivariate normal distribution with covariance matrix determined by the kernel.

The kernel therefore controls the smoothness and structure of functions drawn from the GP prior.

3.4 Example: Squared Exponential Kernel

One of the most widely used kernels is the squared exponential kernel

$$K(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right).$$

The corresponding RKHS contains very smooth functions, in fact infinitely differentiable functions.

The parameter ℓ determines the length scale of variation: larger values of ℓ produce smoother functions.

3.5 Gaussian Process Regression

Suppose we observe data

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2).$$

Let

$$\mathbf{y} = (y_1, \dots, y_n)^\top.$$

Define the kernel matrices

$$K = K(X, X), \quad K_* = K(X, x_*), \quad K_{**} = K(x_*, x_*).$$

The posterior predictive distribution is

$$f(x_*)|y \sim N(\mu_*, \sigma_*^2)$$

where

$$\mu_* = K_*^\top (K + \sigma^2 I)^{-1} y$$

and

$$\sigma_*^2 = K_{**} - K_*^\top (K + \sigma^2 I)^{-1} K_*.$$

3.6 Connection with Kernel Ridge Regression

The posterior mean

$$\hat{f}(x) = K(x, X)^\top (K + \sigma^2 I)^{-1} y$$

coincides with the solution of the optimization problem

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}_K}^2.$$

Thus Gaussian process regression can be interpreted as a Bayesian version of kernel ridge regression.

The RKHS norm $\|f\|_{\mathcal{H}_K}$ acts as a smoothness penalty.

3.7 Interpretation

This duality shows that

$$\text{Gaussian process priors} \quad \longleftrightarrow \quad \text{RKHS regularization.}$$

From a Bayesian perspective we place a prior on functions through the Gaussian process.
 From a frequentist perspective, we estimate functions in an RKHS using penalized least squares.
 Remarkably, the resulting estimator is the same.

3.8 References

For further reading see

- Rasmussen and Williams (2006), *Gaussian Processes for Machine Learning*
- Wahba (1990), *Spline Models for Observational Data*
- Murphy (2012), *Machine Learning: A Probabilistic Perspective*

```
library(MASS)

# squared exponential kernel
kernel <- function(x1, x2, l = 0.3, sigma = 1){
  sigma^2 * exp(-(outer(x1,x2,"-")^2)/(2*l^2))
}

# input grid
x <- seq(0,1,length=100)

# covariance matrix
K <- kernel(x,x)

# sample functions
set.seed(1)
f <- mvrnorm(3, mu = rep(0,100), Sigma = K)

matplot(x, t(f), type="l", lty=1,
        main="Samples from Gaussian Process",
        xlab="x", ylab="f(x)")

# training data
x_train <- seq(0,1,length=10)
y_train <- sin(2*pi*x_train) + rnorm(10,0,0.1)

x_test <- seq(0,1,length=100)

sigma_noise <- 0.1

K <- kernel(x_train,x_train)
K_s <- kernel(x_train,x_test)
K_ss <- kernel(x_test,x_test)

# posterior mean
mu <- t(K_s) %*% solve(K + sigma_noise^2*diag(10)) %*% y_train

# posterior covariance
cov_post <- K_ss - t(K_s) %*% solve(K + sigma_noise^2*diag(10)) %*% K_s
```

```
sd <- sqrt(diag(cov_post))

plot(x_test, mu, type="l", lwd=2,
      ylim=c(-2,2),
      main="Gaussian Process Regression")

lines(x_test, mu+2*sd, lty=2)
lines(x_test, mu-2*sd, lty=2)
points(x_train, y_train)
```

4 Hilbert Spaces and Function Spaces

Machine learning algorithms often optimize over spaces of functions. To do this systematically, we need a notion of distance, angles, and convergence between functions.

Hilbert spaces provide exactly this structure.

Definition 4.1 (Hilbert Space). A Hilbert space $(\mathcal{H}, \langle \cdot, \cdot \rangle)$ is a vector space equipped with an inner product such that every Cauchy sequence converges in the space.

The inner product defines a norm:

$$\|f\|_{\mathcal{H}} = \sqrt{\langle f, f \rangle}.$$

This allows us to measure:

- size of functions,
- distance between functions,
- angles between functions.

Examples of Inner Products in Hilbert Spaces

The concept of a Hilbert space generalizes the familiar dot product geometry of $\mathbb{R}^p$ to infinite-dimensional spaces.

1. Euclidean space $\mathbb{R}^p$

For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^p$,

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{j=1}^p x_j y_j.$$

This inner product produces the usual Euclidean norm

$$\|\mathbf{x}\| = \left(\sum_{j=1}^p x_j^2 \right)^{1/2}.$$

The space $\mathbb{R}^p$ with this inner product is the simplest example of a Hilbert space.

2. Square-integrable functions $L^2(\Omega)$

For functions f, g on a domain Ω ,

$$\langle f, g \rangle = \int_{\Omega} f(x)g(x) dx.$$

The induced norm is

$$\|f\| = \left(\int_{\Omega} f(x)^2 dx \right)^{1/2}.$$

The space $L^2(\Omega)$ is a fundamental infinite-dimensional Hilbert space.

3. Sequence space ℓ^2

For sequences

$$x = (x_1, x_2, \dots), \quad y = (y_1, y_2, \dots),$$

define

$$\langle x, y \rangle = \sum_{j=1}^{\infty} x_j y_j,$$

provided

$$\sum_{j=1}^{\infty} x_j^2 < \infty.$$

The space ℓ^2 can be viewed as an infinite-dimensional analogue of $\mathbb{R}^p$.

4. Sobolev space H^1

For sufficiently smooth functions,

$$\langle f, g \rangle = \int f(x)g(x) dx + \int f'(x)g'(x) dx.$$

This inner product measures both function size and smoothness.

Sobolev spaces play an important role in smoothing splines and penalized regression.

Key idea:

A Hilbert space is simply a vector space where an inner product allows geometry (lengths and angles) to be defined, and where limits of Cauchy sequences remain inside the space.

5. Hölder spaces (not Hilbert spaces)

For $\alpha \in (0, 1]$, the Hölder space $C^\alpha(\Omega)$ consists of functions satisfying

$$\|f\|_{C^\alpha} = \sup_{x \in \Omega} |f(x)| + \sup_{x \neq y} \frac{|f(x) - f(y)|}{|x - y|^\alpha} < \infty.$$

This norm measures smoothness in terms of local regularity.

Although Hölder spaces are complete normed spaces (Banach spaces), they generally do **not** admit a natural inner product satisfying

$$\|f\| = \sqrt{\langle f, f \rangle}.$$

Therefore Hölder spaces are typically **not Hilbert spaces**.

Interpretation:

Hölder spaces measure local smoothness, while Hilbert spaces measure smoothness through quadratic averages. Kernel methods and RKHS theory rely on Hilbert-space structure, which is why Sobolev and RKHS spaces are preferred in learning theory.

Completeness is important because optimization algorithms often generate sequences of approximations. Completeness guarantees that the limit remains inside the space.

Hilbert spaces generalize Euclidean geometry to infinite dimensions. Hilbert spaces provide a geometric framework for working with functions, just as Euclidean space provides a geometric framework for vectors.

5 Reproducing Kernel Hilbert Spaces (RKHS)

Hilbert spaces of functions are very general. However, in machine learning we need an additional property: we must be able to evaluate functions at specific input points.

Definition 5.1 (Reproducing Kernel Hilbert Space). Let $\mathcal{X}$ be a set and let $\mathcal{H}$ be a Hilbert space of functions $f : \mathcal{X} \rightarrow \mathbb{R}$ (or $\mathbb{C}$). The space $\mathcal{H}$ is called a *reproducing kernel Hilbert space (RKHS)* if for every $x \in \mathcal{X}$ the point evaluation functional

$$L_x : \mathcal{H} \rightarrow \mathbb{R}, \quad L_x(f) = f(x),$$

is a continuous linear functional.

Equivalently, $\mathcal{H}$ is an RKHS if there exists, for each $x \in \mathcal{X}$, a function $K_x \in \mathcal{H}$ such that for all $f \in \mathcal{H}$

$$f(x) = \langle f, K_x \rangle_{\mathcal{H}}.$$

The function $K_x(\cdot)$ is called the *reproducing kernel at x* . The associated kernel function

$$K(x, x') = K_{x'}(x)$$

is called the *reproducing kernel of the space*.

Reproducing property. It is common to write the function K_x as $K(\cdot, x)$. Thus for each $x \in \mathcal{X}$, the function

$$K(\cdot, x) \in \mathcal{H}$$

belongs to the RKHS and acts as a representer of evaluation at x . The defining property of the RKHS is the *reproducing property*

$$f(x) = \langle f, K(\cdot, x) \rangle_{\mathcal{H}}, \quad f \in \mathcal{H}.$$

This identity means that evaluating a function at a point x can be written as an inner product in the Hilbert space. In other words, the function $K(\cdot, x)$ plays the role of a feature or probe that extracts the value of f at location x .

Example. Consider the kernel

$$K(x, x') = 1 + xx'$$

on $\mathcal{X} = \mathbb{R}$. The associated RKHS consists of functions of the form

$$f(x) = a + bx,$$

with inner product

$$\langle f, g \rangle_{\mathcal{H}} = a_f a_g + b_f b_g.$$

For a fixed point x , the kernel function is

$$K(\cdot, x) = 1 + x(\cdot).$$

Let

$$f(t) = a + bt.$$

Then

$$\langle f, K(\cdot, x) \rangle_{\mathcal{H}} = \langle a + bt, 1 + xt \rangle_{\mathcal{H}} = a + bx = f(x).$$

Thus, the inner product with $K(\cdot, x)$ reproduces the value of the function at x , illustrating the reproducing property.

Also,

$$\langle K(\cdot, x), K(\cdot, x') \rangle_{\mathcal{H}} = \langle a + xt, 1 + x't \rangle_{\mathcal{H}} = 1 + xx' = K(x, x').$$

You can

Interpretation

The reproducing property

$$f(x) = \langle f, K(\cdot, x) \rangle_{\mathcal{H}}$$

has several useful interpretations:

- Evaluating a function at a point x is equivalent to taking an inner product with the kernel function $K(\cdot, x)$ in the Hilbert space $\mathcal{H}$.
- The function $K(\cdot, x)$ acts like a “basis function centered at x ” that probes the value of f at that location.
- Functions in the RKHS can be constructed from linear combinations of kernel functions

$$f(\cdot) = \sum_{i=1}^n \alpha_i K(\cdot, x_i).$$

- As a consequence, many learning algorithms in RKHS reduce to estimating the coefficients $\{\alpha_i\}$ associated with these kernel functions.

Thus, kernels play a role similar to basis functions in regression.

5.1 Positive Definite Kernels

Not every function $k(\mathbf{x}, \mathbf{x}')$ defines an RKHS.

A function k is a valid kernel if and only if

$$\sum_{i,j=1}^n \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j) \geq 0 \quad \text{for all } \alpha_1, \dots, \alpha_n.$$

Such kernels are called **positive definite**.

Positive definiteness ensures that kernel matrices

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$

are always nonnegative definite.

This guarantees that optimization problems involving kernels have stable solutions.

Common Kernels

- **Linear kernel**

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'.$$

This reproduces ordinary linear regression.

- **Polynomial kernel**

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d.$$

This corresponds to polynomial regression in a high-dimensional space.

- **Gaussian (RBF) kernel**

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right).$$

This kernel produces very smooth functions and is widely used in machine learning.

Key insight:

Choosing a kernel is equivalent to choosing a function space in which learning takes place.

6 Learning in RKHS and the Representer Theorem

Consider the regularized risk minimization problem:

$$\min_{f \in \mathcal{H}} \left\{ \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

Theorem 6.1 (Representer Theorem). *Any minimizer $f^* \in \mathcal{H}$ admits a finite expansion:*

$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x}).$$

Remark 6.1. This result is fundamental:

- Optimization over infinite-dimensional $\mathcal{H}$
- Reduces to finite-dimensional coefficients α
- Explains why kernel methods scale with n

7 Gaussian RKHS and Smoothness

The Gaussian (RBF) kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

induces an RKHS of extremely smooth (analytic) functions.

Properties:

- Infinite-dimensional
- Universal approximator on compact sets
- Strong smoothness control via $\|f\|_{\mathcal{H}}$

8 Radial Basis Function (RBF) Networks

8.1 Definition

An RBF network has the form:

$$f(\mathbf{x}) = \sum_{j=1}^M w_j \phi(\|\mathbf{x} - \mathbf{c}_j\|),$$

where:

- $\phi(r) = \exp(-r^2/(2\sigma^2))$ (Gaussian)
- $\mathbf{c}_j$ are centers
- w_j are weights

8.2 RBF Networks as Kernel Expansions

If we set $\mathbf{c}_j = \mathbf{x}_j$ (training points), then:

$$f(\mathbf{x}) = \sum_{j=1}^n \alpha_j k(\mathbf{x}_j, \mathbf{x}),$$

which is exactly the RKHS representation from the Representer Theorem.

Remark 8.1. Key insight:

RBF networks are finite-dimensional approximations to Gaussian RKHS regression.

Differences:

	RKHS Kernel Method	RBF Network
Centers	All data points	Learned or fixed subset
Dimension	n	$M \ll n$
Training	Convex	Often nonconvex

9 RBF Networks as Shallow Neural Networks

An RBF network is a 2-layer neural network:

$$\mathbf{x} \rightarrow \phi(\|\mathbf{x} - \mathbf{c}_j\|) \rightarrow \sum_j w_j \phi(\cdot).$$

Characteristics:

- Nonlinear hidden layer
- Linear output layer
- Localized activations

Training approaches:

- Fix centers (e.g. k-means), optimize weights
- Jointly optimize centers and weights

10 Neural Networks vs RKHS

10.1 Shallow Networks

For fixed hidden-layer parameters:

$$f(\mathbf{x}) = \sum_{j=1}^M a_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

is a linear model in feature space. Thus shallow networks behave like kernel machines with learned kernels.

10.2 Deep Networks

Deep neural networks differ fundamentally:

- Representation is hierarchical
- Feature space is data-adaptive
- Optimization is nonconvex

However:

- Neural Tangent Kernel (NTK) shows infinite-width networks converge to kernel regression
- Gaussian process limits link deep nets to kernel methods

Remark 10.1. In the infinite-width limit:

$$\text{Neural network} \approx \text{kernel machine}.$$

Finite-width networks interpolate between kernels and adaptive feature learning.

Why RKHS theory is useful

Understanding RKHS theory helps explain several fundamental aspects of modern learning methods.

Why kernels work

Kernels implicitly define feature maps

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle,$$

so learning with kernels is equivalent to performing linear regression in a possibly infinite-dimensional feature space:

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

RKHS theory formalizes this idea by identifying the function space $\mathcal{H}$ in which learning takes place.

Why regularization is necessary

Kernel methods typically optimize

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \lambda \|f\|_{\mathcal{H}}^2.$$

Without the penalty term $\|f\|_{\mathcal{H}}^2$, the function space is often too large and perfect interpolation becomes possible. Regularization controls the size and smoothness of f and ensures stable estimation.

Why solutions often have simple finite representations

Although optimization takes place over an infinite-dimensional space, the Representer Theorem shows that the solution has the form

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x}).$$

Thus infinite-dimensional optimization reduces to estimating a finite vector of coefficients.

How classical smoothing methods relate to modern machine learning

Many classical smoothing methods are special cases of RKHS learning.

For example:

- Smoothing splines solve

$$\min_f \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2 + \lambda \int (f''(x))^2 dx,$$

which is an RKHS optimization problem.

- Kernel regression and Gaussian process regression are also RKHS methods with different kernels.

Key insight:

RKHS theory shows that many classical statistical methods and modern machine learning algorithms are instances of the same general principle: learning by regularized optimization in a function space.

11 Unifying View

- RKHS: optimization in function space with explicit regularization
- Kernel methods: implicit feature maps, convex optimization
- RBF networks: finite kernel expansions
- Neural networks: learned feature maps + nonconvex optimization

Conceptual ladder:

RKHS $\Rightarrow$ Kernel methods $\Rightarrow$ RBF networks $\Rightarrow$ Neural networks.

12 Takeaways

- RKHS theory provides mathematical rigor for learning in function spaces
- Representer theorem explains why kernels work
- RBF networks are kernel machines with learned sparsity
- Neural networks generalize kernels by learning representations
- Modern theory connects deep learning back to RKHS via NTK and GP limits

Final message:

RKHS theory is not an alternative to neural networks-it is their mathematical ancestor.

References

- Youngmin Cho and Lawrence Saul. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems*, 2009.
- Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. *International Conference on Learning Representations*, 2018.
- Radford Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.
- Christopher K. I. Williams. Computation with infinite neural networks. *Neural Computation*, 10(5):1203–1216, 1997.